

การเปรียบเทียบสมรรถนะกับงานวิจัยที่ตีพิมพ์แล้ว

CyberCase Intelligence Framework เทียบกับ TechniqueRAG, H-TechniqueRAG และ TTP-LM บนชุดข้อมูลและตัวชี้วัดเดียวกัน

สาขาที่วัด: การระบุเทคนิค MITRE ATT&CK จากข้อความข่าวกรองภัยคุกคาม · branch `eval/published-baselines`

ข้อค้นพบหลัก: ผลของการค้นคืนขึ้นอยู่กับว่าโมเดลกำเนิดข้อความรู้คำตอบอยู่ก่อนหรือไม่ เมื่อจับคู่กับโมเดลที่น่าจะเคยเห็นชุดทดสอบนี้มาก่อน การค้นคืนลด F1 ลง 0.068 แต่เมื่อจับคู่กับโมเดลที่ไม่มีความรู้เดิมนั้น การค้นคืนเพิ่ม F1 ขึ้น 0.269 หรือ **+89%** วัดบนตัวอย่างชุดเดียวกัน ด้วยการค้นคืนชุดเดียวกัน ต่างกันเพียงโมเดลกำเนิดข้อความ

1. เหตุผลและที่มา

อาจารย์ที่ปรึกษาเสนอว่า การวัดผลระบบด้วยชุดข้อมูลที่สร้างขึ้นเองไม่เพียงพอต่อการยืนยันคุณภาพ เพราะไม่มีจุดอ้างอิงภายนอก จึงควรทดสอบด้วย **ชุดข้อมูลและตัวชี้วัดชุดเดียวกับงานวิจัยที่ตีพิมพ์แล้ว**

งานวิจัยที่เลือกเป็นคู่เทียบ

ชื่อย่อ	งานวิจัย	ชุดข้อมูล	ตัวชี้วัด
TTP-LM	Advancing TTP Analysis: Encoder-Only and Decoder-Only LMs with RAG (arXiv 2401.00280)	MITRE procedure examples	Precision, Recall
TechniqueRAG	TechniqueRAG: Retrieval Augmented Generation for Adversarial Technique Annotation (ACL Findings 2025, arXiv 2505.11988)	TRAM, Procedures, Expert	P, R, F1, P@k / R@k
H-TechniqueRAG	Hierarchical RAG for Adversarial Technique Annotation (arXiv 2604.14166)	CTI-RCM, MITRE CTI, TRAM	micro-F1, P, R, MAP@10

ทั้งสามงานทำโจทย์เดียวกัน คือรับข้อความ CTI ภาษาอังกฤษสั้น ๆ แล้วคืนชุดรหัสเทคนิค ATT&CK และ **TRAM** เป็นชุดข้อมูลรวมของทั้งสามงาน จึงถูกเลือกเป็นเป้าหมายหลัก

ขอบเขตที่ต้องระบุให้ชัด

โจทย์ของเขาเป็นการติดป้ายกำกับข้อความสั้น (80–400 ตัวอักษร) ส่วนโจทย์ของเราคือ **สำนวนคดีภาษาไทยขนาดยาว** → **รายงานสรุปเชิงโครงสร้างสำหรับพนักงานอัยการ** สิ่งที่เทียบกันได้จริงคือส่วนการค้นคืนและการระบุเทคนิค ไม่ใช่ผลผลิตปลายทาง

นอกจากนี้ **ทุก arm** ของเราเป็น **zero-shot** ขณะที่ตัวเลขพาดหัวของ TechniqueRAG มาจากโมเดลที่ fine-tune แล้ว การเทียบที่เป็นธรรมจึงต้องเทียบกับแถวที่ไม่ fine-tune เช่นกัน

2. การเตรียมข้อมูล

2.1 ชุดข้อมูล

ดึงจาก HuggingFace `QCRI/TechniqueRAG-Datasets` เฉพาะ split `*_zeroshot_test` เพราะ split แบบ few-shot ฝัง context ที่คั่นมาแล้วไว้ใน prompt ซึ่งเป็นส่วนที่ระบบเราต้องผลิตเอง ใช้วิธีดึงตอนรัน ไม่ผนวกเข้า **repo** เนื่องจากโค้ดต้นทางเป็นสัญญาอนุญาต GPL-3.0

ชุดข้อมูล	จำนวน	ป้ายกำกับ	ความยาว input (median)	สัดส่วน sub-technique
TRAM (ใช้ในรายงานนี้)	725	single-label (87%)	95 ตัวอักษร	23.9%
Expert	157	multi-label (median 3)	406 ตัวอักษร	39.3%
Procedures	1,767	single-label	80 ตัวอักษร	50.7%

2.2 การพิสูจน์ว่าตัวชี้วัดตรงกับต้นทาง

นำฟังก์ชันจาก `evaluate.py` ของต้นทางมาเป็นตัวอ้างอิง (เก็บไว้ใน repo เพื่อไม่ให้ปนสัญญาอนุญาต GPL) แล้วป้อนข้อมูลสุ่ม 400 ชุดเข้าทั้งสองฝั่ง

ผล: Precision, Recall และ F1 ตรงกันทุกชุด คลาดเคลื่อนไม่เกิน 1e-12 ทั้งสองโหมด

ประเด็น	พฤติกรรมของต้นทาง	การจัดการ
การคำนวณ F1	คำนวณจากค่าเฉลี่ยของ P และ R ไม่ใช่เฉลี่ย F1 รายตัวอย่าง	ทำตามทุกประการ และรายงาน micro-F1 เพิ่มเป็นตัวเทียบ
โหมดปรียาย	<code>--mode technique</code> ยูน sub-technique ขึ้นเป็น parent	รายงานทั้งสองโหมด ระบุกำกับเสมอ
MRR	คำนวณหลัง <code>list(set(preds))</code> ซึ่งทำลายลำดับ และ Python สุ่ม hash ต่อ process จึงให้ค่าไม่คงที่	รายงานจากลำดับจริง พร้อมระบุว่าเทียบตรงไม่ได้

2.3 การปรับรุ่น ATT&CK ให้ตรงกัน

ป้ายเฉลี่ยของชุดข้อมูลกำกับไว้ในยุค ATT&CK รวรวุ่น v14 ขณะที่ดัชนีของเราสร้างจาก **v19.0** หากไม่ปรับให้ตรงกันก่อน ระบบจะค้นเจอสิ่งเดียวกันแต่ถูกนับว่าผิดเพราะรหัสไม่ตรง

รหัสในชุดข้อมูล (v14)	สถานะใน v19	ผลลัพธ์
T1562 Impair Defenses	ย้ายไป T1685	แปลงรหัส
T1562.001 – .006	ย้ายไป T1685 / T1685.001 / T1686 / T1690	แปลงรหัส
T1574.002 DLL Side-Loading	ยุบรวมเป็น T1574.001 (DLL)	แปลงรหัส
T1070.001 / .002	ย้ายไป T1685.005 / T1685.006	แปลงรหัส
T1043, T1064	ยกเลิกโดยไม่มีรหัสทดแทน	ตัดตัวอย่างทิ้ง (12 ตัวอย่าง)

ตารางแปลงสร้างจากความสัมพันธ์ `revoked-by` ในไฟล์ STIX ที่มีอยู่ใน repo ได้ **196** คู่ จึงทำซ้ำได้ ไม่ใช่การแก้ด้วยมือ หลังการแปลง ป้ายเฉลี่ยทุกรายการค้นคืนได้ครบ **100%** TRAM เหลือตัวอย่างที่วัดได้ **713** จาก **725**

3. การพัฒนาชุดทดสอบ

3.1 arm ที่ใช้วัด

หลักการที่ยึด: ตัวเลขที่อ้างว่าเป็นคะแนนของระบบ ต้องมาจาก **pipeline** เดียวกันที่ใช้จริงบน **production** arm อื่น เป็น **ablation** มีไว้อธิบายว่าทำไมตัวเลขนั้นถึงออกมาเช่นนั้น

arm	retrieval	LLM	สถานะ	ใช้ตอบคำถาม
agent	ครบ	ครบ	headline ยังไม่ได้รัน	ระบบที่ให้บริการจริงทำได้เท่าไร
retrieval	มี	ไม่มี	ablation	คุณภาพการค้นคืนล้วน ๆ
rag-en	มี	มี	ablation	ค้นคืนแล้วให้ LLM สรุป
llm-only	ไม่มี	มี	ablation	โมเดลตอบเองได้เท่าไรโดยไม่มีเรา

3.2 การตั้งค่าและองค์ประกอบ เทียบกับ TechniqueRAG

องค์ประกอบ	ของเรา	TechniqueRAG
Embedding	BAAI/bge-m3 · 1024-dim · dense+sparse	ไม่มี — BM25 เป็น lexical ล้วน
Retriever	Qdrant hybrid · RRF · top-K 20	BM25 · K=40
Re-ranker	bge-reranker-v2-m3 · 559M · ในเครื่อง	DeepSeek v3 · 671B total / 37B active · เรียก API
Generator	Qwen3.5-9B · zero-shot	Ministral 8B · LoRA fine-tune
Vector DB	Qdrant Cloud · 23,542 points	ไม่ต้องใช้
Graph DB	Neo4j · 2,195 nodes · 23,465 edges · 2 hops	ไม่มี
คลังที่ค้น	MITRE ATT&CK v19 (สาธารณะ)	ตัวอย่างติดเฉลย 3,469 รายการ จาก train split
ภาษา	ไทย + อังกฤษ	อังกฤษเท่านั้น

ความต่างที่สำคัญที่สุดคือแกลสียี่ห้อ TechniqueRAG ไม่ได้ค้น “ความรู้” แต่ค้น ตัวอย่างที่มนุษย์ติดเฉลยไว้แล้ว จาก distribution เดียวกับชุดทดสอบ (exemplar-based RAG) ขณะที่ระบบนี้ค้นฐานความรู้สาธารณะ (knowledge-based RAG) วิธีของเขาจึงใช้ไม่ได้กับโดเมนที่ไม่มีข้อมูลติดป้ายกำกับ ซึ่งเป็นกรณีของสำนวนคดีไทย

3.3 ข้อบกพร่องที่พบและแก้ระหว่างการพัฒนา

ข้อบกพร่อง	อาการ	การแก้ไข
node_label_filter ไม่ทำงาน	รับพารามิเตอร์แล้วไม่ส่งต่อ ผู้เรียกได้ผลที่ไม่ถูกกรองโดยไม่มี การแจ้งเตือน	ต่อพารามิเตอร์ให้ครบสาย รองรับหลาย label
Qdrant ปฏิเสธการกรอง	ล้มทั้ง 713 ตัวอย่างด้วย 400 Index required but not found	เปลี่ยนไปกรองฝั่งไคลเอนต์ ไม่แต่ฐานข้อมูลที่ใช้ร่วมกัน
context ถูกตัดเหลือ 5	FINAL_TOP_K = 5 ทำให้ LLM เห็น candidate 5 จาก 20	สร้างตัวประกอบ context เฉพาะการวัดผล ให้เห็นครบ
โมเดลคิดจนหมดโควตา	Qwen3.5-9B เป็น reasoning model ใช้ 4,096 token ไปกับการคิด แล้วไม่เขียนคำตอบ	ปิด thinking — ถูกลง 115 เท่า และเป็นการเทียบที่แฟร์กับ Ministral 8B ที่ไม่ใช่ reasoning model
การรันค้างการ	ไม่ได้ตั้ง timeout ให้ client · 1 call ค้าง บล็อกทั้ง batch · CPU นิ่งสนิท 5 นาที	timeout 120s ต่อ request + 180s ต่อ batch ทั้งตัวที่ค้างแล้วให้ resume เก็บ
การรวมคะแนนข้าม collection	min-max normalize แยกแต่ละ collection บังคับให้อันดับ 1 ของแต่ละฝั่งเป็น 1.0 เสมอ ไม่ว่าจะเกี่ยวข้องจริงหรือไม่ ผลคือ candidate ที่เป็น technique เหลือเพียง 7.9 จาก 20 และ 6 ตัวอย่างไม่ได้ technique เลย	เปลี่ยนเป็นรวมด้วยอันดับ weight / (RRF_K + rank) พร้อมน้ำหนักที่ประกาศไว้ชัดเจน

4. ผลการวัด

ชุดข้อมูล TRAM · 713 ตัวอย่าง · zero-shot ทุก arm

4.1 ผลของการค้นคืน แยกตามโมเดลกำเนิดข้อความ — ระดับ technique

โมเดล	llm-only (ไม่มี retrieval)	rag-en (มี retrieval)	ผลของ retrieval	เชิงสัมพัทธ์
gpt-5.6-luna	0.693	0.625	-0.068	-10%
qwen3.5-9b	0.302	0.571	+0.269	+89%

4.2 ระดับ sub-technique

โมเดล	llm-only	rag-en	ผลของ retrieval	เชิงสัมพัทธ์
gpt-5.6-luna	0.515	0.460	-0.055	-11%
qwen3.5-9b	0.237	0.424	+0.187	+79%

4.3 ตารางรวมทุก arm

arm	P	R	F1	P@1	P@3
ระดับ technique					
llm-only · luna	0.653	0.738	0.693	0.687	0.653
rag-en · luna	0.589	0.665	0.625	0.621	0.589
rag-en · qwen3.5-9b	0.495	0.675	0.571	0.582	0.519
llm-only · qwen3.5-9b	0.260	0.359	0.302	0.288	0.273
retrieval — กรองเฉพาะ technique	0.070	0.805	0.129	0.373	0.251
retrieval — ไม่กรอง	0.145	0.667	0.238	0.384	0.263
ระดับ sub-technique					
llm-only · luna	0.484	0.551	0.515	0.512	0.485
rag-en · luna	0.429	0.495	0.460	0.454	0.429
rag-en · qwen3.5-9b	0.351	0.537	0.424	0.428	0.364
llm-only · qwen3.5-9b	0.195	0.302	0.237	0.215	0.206

4.4 เียบกับตัวเลขที่ตีพิมพ์ — TRAM ระดับ technique

ระบบ	ประเภท	P	R	F1
NCE	retriever	0.903	0.789	0.842
TechniqueRAG	fine-tuned	0.760	0.721	0.740
BM25	retriever	0.679	0.647	0.663
IntelEx	RAG	0.607	0.707	0.653
DeepSeek v3 (RAG)	RAG	0.546	0.774	0.640
TechniqueRAG re-ranker	retriever	0.647	0.614	0.630
ของเรา — rag-en · luna	RAG	0.589	0.665	0.625
GPT-4 (RAG)	RAG	0.555	0.706	0.622
RankGPT	retriever	0.619	0.586	0.602
ของเรา — rag-en · qwen3.5-9b	RAG	0.495	0.675	0.571
 Ministral 8B (RAG) — คู่เทียบที่คุมตัวแปร	RAG	0.519	0.576	0.546
DeepSeek v3	zero-shot	0.437	0.657	0.525
GPT-4o	zero-shot	0.383	0.500	0.434
ของเรา — llm-only · qwen3.5-9b	zero-shot	0.260	0.359	0.302
Ministral 8B	zero-shot	0.077	0.317	0.124

แถว retriever ของงานตีพิมพ์รายงาน $P \approx R$ บนชุด single-label ซึ่งบ่งชี้ว่าให้คำตอบราวหนึ่งรายการต่อตัวอย่าง ค่าที่เทียบได้จากฝั่งเราจึงเป็น $P@1$ ไม่ใช่ precision ของลิสต์เต็ม

4.5 การทดลองปรับปรุงตัวค้นคืน

ตัวค้นคืนพบเฉลี่ยใน 20 อันดับแรกถึง 83.3% แต่จัดให้อยู่อันดับ 1 ได้เพียง 37.3% คอขวดจึงอยู่ที่การจัดอันดับ ไม่ใช่การค้นหา จึงทดลองแก้ที่สิ่งที่บ่อนให้ตัวจัดอันดับ สามทาง

การทดลอง	P@1	P@3	recall	candidate ต่อคำถาม	ผลลัพธ์ว่าง
เส้นฐาน — กรองเฉพาะ technique	0.373	0.251	0.805	19.9	0
ล้าง corpus (ตัดขยะ 18.5%)	0.363	0.251	0.812	19.9	0
เส้นฐาน — ไม่กรอง	0.384	0.263	0.667	7.9	6
แก้การรวมคะแนนเป็นแบบอันดับ	0.388	0.259	0.725	12.1	1

การสร้าง corpus ตัดข้อความออก 22.1% ของทั้งคลัง (เอกสาร technique 17.4% · median 1,252 → 1,026 ตัวอักษร) โดยเก็บ anchor text ของลิงก์ และเนื้อใน `<code>` ไว้ เพราะเส้นทางรีจิสทรีเป็นสตริงที่คำอธิบายเชิงกระบวนการอ้างถึงตรง ๆ สร้างเป็น collection แยก ไม่แตะดัชนีที่ใช้งานจริง

4.6 การทดลองปิดตัวจัดอันดับใหม่

เมื่อการแก้ที่ฝั่ง input ทั้งสามทางไม่ให้ผล จึงทดสอบตรง ๆ ว่า cross-encoder มีส่วนช่วยหรือไม่ โดยรันด้วยลำดับดิบจาก Qdrant (hybrid dense+sparse) ไม่ผ่านการจัดอันดับใหม่เลย

การตั้งค่า	P@1	P@3	R@3	recall
ใช้ bge-reranker-v2-m3	0.373	0.251	0.551	0.805
ไม่ใช้ตัวจัดอันดับใหม่	0.396	0.255	0.540	0.805

recall เท่ากันทุกประการ เพราะชุด candidate เหมือนกัน ต่างกันเพียงลำดับ

5. สรุปผล

ข้อค้นพบที่ 1 — ผลของการค้นคืนกลับทิศทางโมเดลที่จับคู่ด้วย

เป็นข้อค้นพบหลักของงานนี้ เดิมเมื่อวัดด้วย gpt-5.6-luna เพียงโมเดลเดียว ผลออกมาว่าการค้นคืนลดคะแนน ซึ่งเป็นข้อสรุปที่อธิบายได้ยาก เมื่อเพิ่มโมเดลที่สองเข้ามาในการทดลอง ภาพเปลี่ยนทั้งหมด

	llm-only	rag-en	ผลของ retrieval
gpt-5.6-luna	0.693	0.625	-0.068
qwen3.5-9b	0.302	0.571	+0.269 (+89%)

ตัวอย่างชุดเดียวกัน การค้นคืนชุดเดียวกัน prompt เดียวกัน ต่างกันเพียงโมเดลกำเนิดข้อความ ข้อสรุปที่ถูกต้องจึงไม่ใช่ “RAG ทำให้แย่ลง” แต่คือ การค้นคืนช่วยเมื่อโมเดลไม่รู้คำตอบ และรบกวนเมื่อโมเดลรู้ดีกว่าตัวค้นคืน ซึ่งงานนี้วัดได้ทั้งสองทิศทางในการทดลองเดียวกัน

ข้อค้นพบที่ 2 — หลักฐานสนับสนุนการปนเปื้อนของชุดทดสอบ

gpt-5.6-luna ตอบเองได้ 0.693 บนโจทย์ที่ qwen3.5-9b ทำได้ 0.302 — ต่างกัน **0.39** ซึ่งใหญ่เกินกว่าจะอธิบายด้วยขนาดโมเดลเพียงอย่างเดียว และยิ่งสูงกว่าแถว GPT-4o zero-shot ของงานตีพิมพ์ (0.434) ถึง 26 จุด

procedure examples ของ MITRE เป็นข้อมูลสาธารณะ และ luna เป็นโมเดลรุ่นปี 2026 จึงมีความเป็นไปได้สูงว่าเคยเห็นชุดข้อมูลนี้ระหว่างการฝึก ข้อสังเกตนี้กระทบการตีความทุกงานที่วัดผลบน TRAM ด้วยโมเดลรุ่นใหม่ ไม่ใช่เฉพาะงานนี้

ข้อค้นพบที่ 3 — เทียบเท่าคู่เทียบที่คุมตัวแปร ด้วยองค์ประกอบที่เบากว่าเดิม

ที่ระดับ technique ระบบนี้ทำได้ **F1 0.571** เทียบกับ **0.546** ของ Ministral 8B (RAG) ซึ่งเป็นแถวที่คุมตัวแปรรวมได้มากที่สุด: generator ขนาดใกล้เคียงกัน (9B เทียบ 8B) แบบสำเร็จรูปทั้งคู่ ไม่ fine-tune ทั้งคู่ ที่ระดับ sub-technique ได้ 0.424 เทียบกับ 0.377 (+12% เชิงสัมพัทธ์)

งานตีพิมพ์ระบุชัดว่าแถวดังกล่าวไม่ผ่านการ fine-tune — “GPT-4o (RAG), DeepSeek v3 (RAG), and Ministral 8B (RAG) use retrieved exemplars but receive no fine-tuning”

ผลเทียบเท่านี้ได้มาโดยใช้ **re-ranker ขนาด 559M** พารามิเตอร์ที่ประมวลผลในเครื่อง แทน DeepSeek v3 (671B total / 37B active) ที่ต้องเรียกผ่าน API ทุกครั้ง และไม่ต้องอาศัยตัวอย่างที่มีป้ายกำกับในโดเมนจำนวน **3,469** รายการ ซึ่งคลังค้นคืนของเขาพึ่งพา

ระยะห่างที่ระดับ technique คือ 0.025 ซึ่งแคบ จึงควรใช้คำว่า “เทียบเท่า” (comparable) มิใช่ “ดีกว่า” (superior)

ข้อค้นพบที่ 4 — ยังห่างจากตัวเลขพาดหัว และช่องว่างอธิบายได้

ระบบนี้ยังต่ำกว่า TechniqueRAG (0.740) อยู่ **0.169** ที่ระดับ technique ช่องว่างนี้สอดคล้องกับ ablation ในงานเดียวกัน ซึ่งรายงานว่าการ **fine-tune generator** เพียงอย่างเดียวให้ผลต่าง **+0.194** บนโมเดลฐานและชุดค้นคืนเดิม (Ministral 8B RAG 0.546 → TechniqueRAG 0.740)

กล่าวคือ ปัจจุบันเดียวที่งานตีพิมพ์ระบุว่ามีผลมากที่สุด มีขนาดใหญ่กว่าช่องว่างทั้งหมดของระบบนี้ และเป็นปัจจัยที่ระบบนี้ยังไม่ได้ใช้

ข้อค้นพบที่ 5 — ตัวค้นคืนอ่อนในฐานะตัวจัดอันดับ แต่แข็งในฐานะตัวสร้างตัวเลือก

P@1 อยู่ที่ 0.373 ต่ำกว่าทุก baseline ที่ตีพิมพ์ แต่เมื่อพิจารณาทั้งลิสต์ 20 อันดับ เฉลยปรากฏใน 83.3% ของตัวอย่าง (recall 0.805) ปัญหาจึงอยู่ที่การเรียงลำดับ ไม่ใช่การค้นหาไม่เจอ ซึ่งอธิบายว่าเหตุใดการเพิ่ม LLM เข้ามาเลือกจากตัวเลือกเหล่านั้นจึงยกคะแนนได้ถึง +89%

ข้อค้นพบที่ 6 — คอขวดอยู่ที่ฟังก์ชันจัดอันดับ ไม่ใช่สิ่งที่ป้อนให้มัน

มีการทดลองแก้ สิ่งที่ป้อนให้ตัวจัดอันดับ สามทาง และ P@1 ไม่ขยับเกินระดับสัญญาณรบกวนในทุกทาง

การทดลอง	สมมติฐานที่ทดสอบ	ผลต่อ P@1
กรองเหลือเฉพาะ technique	ผลลัพธ์ประเภท Group / Software แย่งตำแหน่งต้นตาราง	0.384 → 0.373
ล้าง corpus	ข้อความประกอบที่ไม่ช่วยแยกแยะบดบังเนื้อหาจริง	0.373 → 0.363
แก้การรวมคะแนน	candidate ที่ถูกต้องถูกเบียดออกก่อนถึงขั้นจัดอันดับ	0.384 → 0.388

เมื่อการแก้ที่ฝั่ง input ครบทุกทางแล้วไม่ให้ผล ช่องว่างที่เหลือจึงอยู่ที่ ฟังก์ชันจัดอันดับเอง คือ bge-reranker-v2-m3 มิใช่สิ่งที่ป้อนให้มัน ข้อสรุปนี้ได้มาโดยการตัดตัวเลือกออกทีละข้อด้วยการวัด มิใช่การอนุมาน

การทดลองแก้การรวมคะแนนยังคงมีคุณค่าในตัวเอง แม้ไม่ช่วยเรื่องอันดับ กล่าวคือ candidate ที่เป็น technique เพิ่มจาก 7.9 เป็น 12.1 ต่อคำถาม ตัวอย่างที่ไม่ได้ผลลัพธ์เลยลดจาก 6 เหลือ 1 และ recall เพิ่มจาก 0.667 เป็น 0.725 ซึ่งเป็นการแก้ความถูกต้องของระบบที่ใช้งานจริง

ข้อค้นพบที่ 7 — ตัวจัดอันดับใหม่ไม่เพียงไม่ช่วย แต่ทำให้แย่ลง

การปิด bge-reranker-v2-m3 แล้วใช้ลำดับติดจาก Qdrant ทำให้ P@1 เพิ่มจาก 0.373 เป็น 0.396 โดย recall ไม่เปลี่ยนแปลง กล่าวคือ cross-encoder กำลังดันคำตอบที่ถูกต้องลงจากอันดับบน มิใช่เพียงไม่สามารถยกขึ้นได้

ข้อนี้เปิดลำดับการวินิจฉัยที่เริ่มจากข้อค้นพบที่ 6 การแก้ที่ฝั่ง input สามทางไม่ให้ผล เพราะปัญหาไม่ได้อยู่ที่ input และเมื่อวัดตัวจัดอันดับโดยตรงก็พบว่าเป็นผลลบ

ผลพลอยได้คือต้นทุนการประมวลผล cross-encoder เป็นขั้นตอนที่แพงที่สุดที่ทำงานทุกคำถาม และบนการติดตั้งจริงที่ใช้ CPU เป็นตัวกำหนดเวลาตอบสนองของการค้นคืนทั้งหมด

ยังสรุปให้ระบบที่ใช้งานจริงไม่ได้ ชุดทดสอบนี้เป็นข้อความอังกฤษล้วนล้วน ขณะที่ตัวจัดอันดับนี้ถูกเลือกมาเพราะให้คะแนนคู่ไทย-อังกฤษได้ ซึ่งเป็นความสามารถที่ชุดทดสอบนี้ไม่ได้ทดสอบเลย การจะปิดใช้งานจริงต้องวัดบนชุดข้อมูลภาษาไทยก่อน

ข้อค้นพบที่ 8 — ชุดข้อมูลฝึกที่มีอยู่ทับซ้อนกับชุดทดสอบ 9%

ในการเตรียมข้อมูลสำหรับปรับจูนตัวจัดอันดับ พบว่ากราฟของโครงการมีความสัมพันธ์ USES ที่ชี้ไปยังเทคนิคพร้อมคำอธิบายเชิงกระบวนการอยู่แล้ว **18,383** รายการ ซึ่งเป็นรูปแบบเดียวกับ input ของชุดทดสอบ และมากกว่าคลังที่ TechniqueRAG ค้นคืน (3,469) หัวเท่า

แต่ชุดทดสอบที่ตีพิมพ์ก็สร้างจาก MITRE procedure examples เช่นกัน ทั้งสองฝั่งจึงมาจากแหล่งเดียวกัน เมื่อตรวจสอบพบว่า **1,656 รายการในคลังฝึกทับซ้อนกับ TRAM, Expert หรือ Procedures** (เฉพาะ TRAM คือ 228 จาก 725 ตัวอย่าง หรือ 31.4% โดย 207 รายการมีป้ายเฉลยตรงกันด้วย)

การเทียบด้วยข้อความตรง ๆ ตรวจไม่พบเลย เพราะชุดทดสอบตัดชื่อผู้กระทำออกจากต้นประโยค

```
MITRE : "[TrickBot](https://attack.mitre.org/...) creates a scheduled task
on the system that provides persistence.(Citation: ...)"
TRAM : "creates a scheduled task on the system that provides persistence."
```

จึงต้องเทียบบนข้อความที่ผ่านการทำให้เป็นมาตรฐานแล้ว (ตัด citation และลิงก์ แปลงเป็นตัวพิมพ์เล็ก ตัดเครื่องหมายวรรคตอน) และใช้การ**บรรจุภายใน** มิใช่การเท่ากันทุกประการ รายการที่ถูกตัดออกทั้งหมดบันทึกไว้เป็นหลักฐานตรวจสอบย้อนกลับ

ข้อนี้เป็นข้อควรระวังสำหรับงานใด ๆ ที่ปรับจูนโมเดลด้วยข้อมูล MITRE แล้ววัดผลบนชุดทดสอบที่สร้างจาก MITRE หากไม่ตัดออกก่อน คะแนนที่ได้จะวัดการจดจำ มิใช่ความสามารถ และจะไม่ถ่ายทอดไปยังส่วนคลัสติ่งที่โมเดลไม่เคยเห็น

6. ข้อจำกัดของการทดลองนี้

- ตัวเลขทั้งหมดมาจาก arm ที่เป็น **ablation** ยังไม่ใช่ pipeline ที่ให้บริการจริง arm agent พัฒนาเสร็จแล้วแต่ยังไม่ได้รัน
- วัดบนชุด TRAM เท่านั้น ยังไม่ได้วัด Expert (multi-label) และ Procedures
- ความสามารถหลายภาษาและการใช้กราฟความสัมพันธ์ **ยังไม่ได้ประเมิน** — ชุดทดสอบนี้เป็นภาษาอังกฤษ และ input สั้นเกินกว่าจะมีความสัมพันธ์ระหว่าง entity ให้ใช้ ทั้งสองข้อจึงไม่ควรถูกอ้างเป็นผลของงานนี้
- การเทียบ rag-en กับแถว RAG ของงานตีพิมพ์ ปนตัวแปรทั้งตัวค้นคืนและคลังข้อมูล แยกจากกันไม่ได้
- MRR เทียบกับต้นทางไม่ได้ ด้วยเหตุผลในหัวข้อ 2.2
- ข้อสรุปเรื่องการปนเปื้อนเป็นการอนุมานจากคะแนน มิใช่การตรวจสอบชุดข้อมูลฝึกของโมเดลโดยตรง

7. งานที่ควรทำต่อ

ลำดับ	งาน	เหตุผล	ต้นทุน
1	รัน arm agent	ได้ตัวเลขของ pipeline จริงตามหลักการที่วางไว้ ปัจจุบันรายงานนี้ยังอ้างได้เฉพาะ ablation	~45 นาที · ~\$0.30
2	ทดลองแก้ข้อผิดพลาดใน prompt	เปลี่ยนจาก “ใช้ข้อมูลที่ค้นมาเป็นหลักฐาน” เป็น “ใช้เมื่อเป็นประโยชน์” เพื่อลดการยึดติดตัวเลือกที่ผิด น่าจะช่วยกรณี luna โดยไม่กระทบ qwen หมายเหตุ: การจำกัดจำนวนรายการที่ตอบไม่ใช่ทางแก้ — วัดแล้วพบว่า ตัดลิสต์ที่ $k=1/2/3$ ให้ F1 0.568 / 0.570 / 0.573 เทียบกับลิสต์เต็ม 0.571 คือแบบราบ	~20 นาที · ~\$0.35
3	ปรับจูนตัวจัดอันดับใหม่	ตัวจัดอันดับให้ผลลบ (ข้อค้นพบที่ 7) แต่การเปลี่ยนไปใช้โมเดลใหญ่กว่า จะทำลายข้อโต้แย้งเรื่องน้ำหนักเบาและใช้บน CPU ไม่ได้ ทางที่รักษาข้อโต้แย้งไว้คือปรับจูนตัวเดิมขนาด 559M ด้วยชุดข้อมูล 15,915 รายการที่เตรียมไว้แล้ว (ตัดการทับซ้อนออกแล้ว 1,656 รายการ)	ต้องใช้ GPU
3ก	ขยายหน้าตาของตัวจัดอันดับ	เดิมกำหนด <code>max_length = 512</code> ไว้ตายตัว ทั้งที่โมเดลรองรับได้ถึง 8192 และราว 16% ของเอกสารยาวเกิน 2,000 ตัวอักษร จึงถูกตัดท้ายทิ้งก่อนถึงโมเดล เป็นคำอธิบายหนึ่งของผลลบข้างต้น	ฟรี · กำลังวัด
4	วัดบนชุด Expert	เป็น multi-label จริง และเป็น split เดียวที่มี $P@1 / P@3$ ที่ตีพิมพ์ให้เทียบตรง อีกทั้งเป็นสนามที่การแตกคำถามย่อยควรได้เปรียบ	~15 นาที · ~\$0.05
5	fine-tune เพื่อปิดช่องว่าง 0.169	โครงการมีโมเดล fine-tune อยู่แล้ว หากได้ผลใกล้เคียง +0.194 ตามที่งานตีพิมพ์รายงาน จะอยู่ในระดับเดียวกับ TechniqueRAG ด้วยโมเดลที่เล็กกว่า	เป็นงานแยกอีกก้อน
6	วัดภาษาไทยบนชุด CTI ไทยของโครงการ	เป็นข้อโต้แย้งที่ชุดทดสอบนี้วัดไม่ได้ และเป็นบริบทที่การปนเปื้อนของโมเดลไม่เป็นปัญหา	ต้องประเมินแยก

ข้อมูลประกอบทั้งหมด ได้แก่ ผลการรันรายตัวอย่างทั้ง 6 ชุด ที่มาของข้อมูล และตัวเลขของงานคู่เทียบ อยู่ใน `rag_service/app/RAG/GraphRAG/evaluation/published/` บน branch `eval/published-baselines`